Merge Sort

AlgoExpert · Very Hard · Sorting

The problem

- Stable divide-and-conquer sort using one auxiliary buffer.

Intuition

- Split in half, sort each half, then merge the sorted halves.

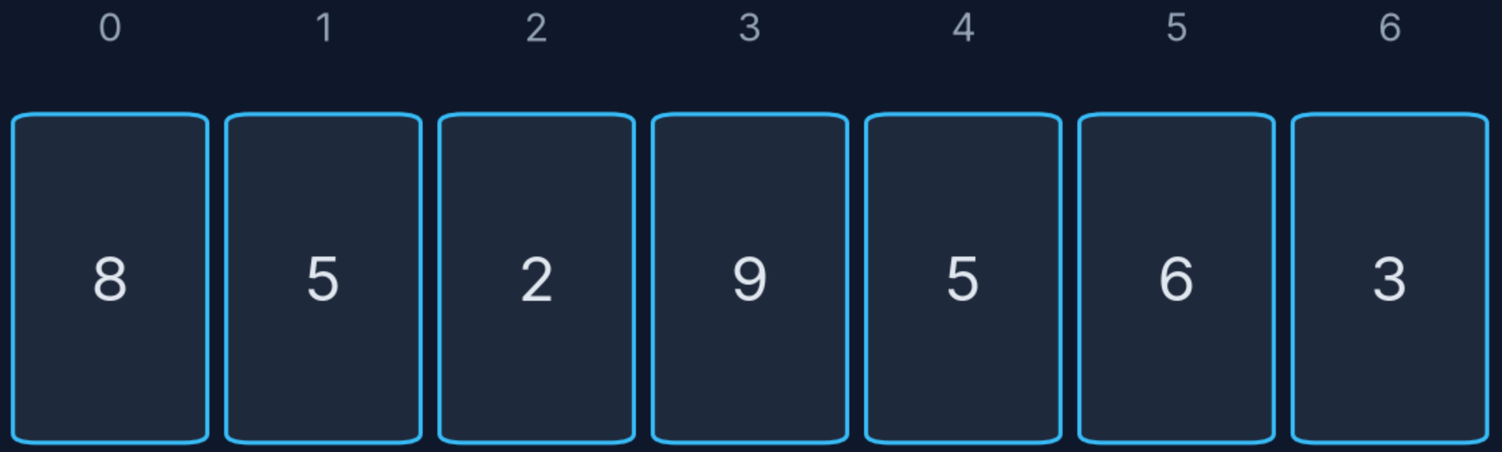
Brute force vs optimal

- Quadratic sorts → $O(n^2)$
- Merge sort → O(n log n), stable
- O(n) auxiliary

Approach

- 1. Recurse on halves
- 2. Merge using a buffer
- 3. Copy back

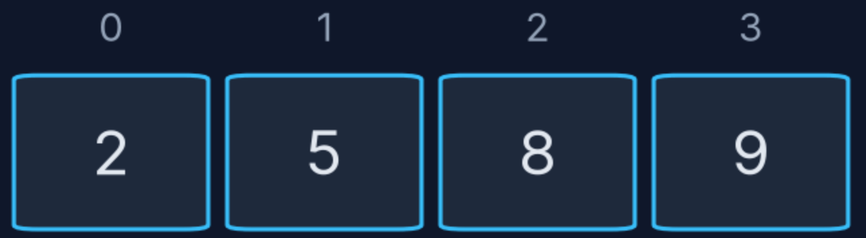
Worked example



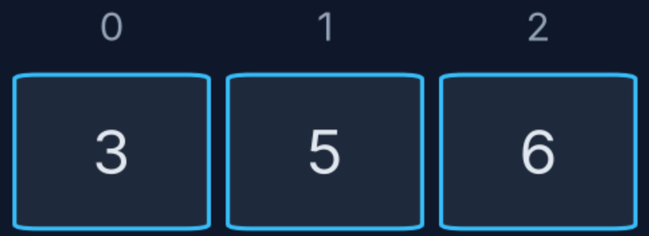
Split in half recursively

Worked example

sorted left



sorted right



Sort both halves

Worked example



Merge with two pointers

Java solution

```java
public class MergeSort {
    public static int[] mergeSort(int[] a) {
        sort(a, new int[a.length], 0, a.length - 1);
        return a;
    }

    private static void sort(int[] a, int[] aux, int lo, int hi) {
        if (lo >= hi) return;
        int mid = (lo + hi) >>> 1;
        sort(a, aux, lo, mid);
        sort(a, aux, mid + 1, hi);
        System.arraycopy(a, lo, aux, lo, hi - lo + 1);
        int i = lo, j = mid + 1;
        for (int k = lo; k <= hi; k++) {
            if (i > mid) a[k] = aux[j++];
            else if (j > hi) a[k] = aux[i++];
            else if (aux[j] < aux[i]) a[k] = aux[j++];
            else a[k] = aux[i++];
        }
    }
}
```

Complexity

- Time: O(n log n)
- Space: O(n)

Edge cases & pitfalls

- Allocate aux once
- ≤ for stability
- Base case size 1

Interview tips & follow-ups

- Compare with quick sort

Recap

- Split in half, sort each half, then merge the sorted halves.
- Complexity: O(n log n)